

FIFO Verification Project

Synchronous FIFO Verification



1.0 VERIFICATION PLAN

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FIFO_1	Constraint to make weighted randomize on rst_n	-	-	A constraint in the package to make weighted Randomization
FIFO_2	Constraint to make weighted randomize on write enable	-	cover the all values for write enable and cross values with wr_ack, overflow,full,empty,almostfull,almostempty and underflow	A constraint in the package to make weighted Randomization
FIFO_3	Constraint to make weighted randomize on read enable	-	cover the all values for read enable and cross values with wr_ack, overflow,full,empty,almostfull,almostempty and underflow	A constraint in the package to make weighted Randomization
FIFO_4	-	-	cover group to cover all values for write enable,read enable,full,empty,almostfull,almostempty,overflow,underflow and write_ack and cover cross values of write enable with read enable and (full,empty,almostfull,almostempty,overflow,underflow and write_ack)	-
FIFO_5	When the reset is asserted, the output values should be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
FIFO_6	randomize inputs	Randomization	-	A checker in the testbench to make sure the output is correct
FIFO_7	trigger an event to give the randomized values for the transaction object in the monitor to sample the values in the coverage package and check data out using a reference model	-	-	A checker in the testbench to make sure the output is correct
FIFO_8	make assertions to cover all outputs and internal signals	-	-	-

2.0 MONITOR

```
1 import shared_pkg::*;
2 import FIFO_transaction_pkg::*;
3 import FIFO_coverage_pkg::*;
4 import FIFO_scoreboard_pkg::*;
5 module fifo_monitor(fifo_if.MONITOR f_if);
6     FIFO_transaction trans_mon = new(65,65);
7     FIFO_coverage cover_mon = new();
8     FIFO_scoreboard score_mon = new();
9     initial begin
10         forever begin
11             wait(sample_event.triggered);
12             @(negedge f_if.clk);
13             trans_mon.data_in = f_if.data_in;
14             trans_mon.rst_n = f_if.rst_n;
15             trans_mon.wr_en = f_if.wr_en;
16             trans_mon.rd_en = f_if.rd_en;
17             trans_mon.data_out = f_if.data_out;
18             trans_mon.wr_ack = f_if.wr_ack;
19             trans_mon.overflow = f_if.overflow;
20             trans_mon.underflow = f_if.underflow;
21             trans_mon.full = f_if.full;
22             trans_mon.almostfull = f_if.almostfull;
23             trans_mon.empty = f_if.empty;
24             trans_mon.almostempty = f_if.almostempty;
25         fork
26             cover_mon.sample_data(trans_mon);
27             score_mon.check_data(trans_mon);
28         join
29
30         if (test_finished == 1'b1) begin
31             $display("correct count =%d , error count =%d",correct_count,error_count);
32             $stop;
33         end
34     end
35 end
36 end
37 endmodule
```

3.0 COVERAGE PACKAGE

```
1 package FIFO_coverage_pkg;
2 import shared_pkg::*;
3 import FIFO_transaction_pkg::*;
4 FIFO_transaction F_cvg_txn = new(70,70);
5
6 class FIFO_coverage;
7     //FIFO_4
8     covergroup cvr_gp;
9         cp_wr_en      : coverpoint F_cvg_txn.wr_en;
10        cp_rd_en      : coverpoint F_cvg_txn.rd_en;
11        cp_full       : coverpoint F_cvg_txn.full;
12        cp_empty      : coverpoint F_cvg_txn.empty;
13        cp_almostfull : coverpoint F_cvg_txn.almostfull;
14        cp_almostempty : coverpoint F_cvg_txn.almostempty;
15        cp_overflow    : coverpoint F_cvg_txn.overflow;
16        cp_underflow   : coverpoint F_cvg_txn.underflow;
17        cp_wr_ack      : coverpoint F_cvg_txn.wr_ack;
18
19        wr_ack_C: cross cp_wr_en, cp_rd_en, cp_wr_ack{
20            illegal_bins zero_zero_one = binsof(cp_wr_en) intersect {0} && binsof(cp_wr_ack) intersect {1};
21        }
22        overflow_C: cross cp_wr_en, cp_rd_en, cp_overflow{
23            illegal_bins zero_w_one = binsof(cp_wr_en) intersect {0} && binsof(cp_overflow) intersect {1};
24        }
25        full_C: cross cp_wr_en, cp_rd_en, cp_full{
26            illegal_bins one_r_one = binsof(cp_rd_en) intersect {1} && binsof(cp_full) intersect {1};
27        }
28        empty_C: cross cp_wr_en, cp_rd_en, cp_empty;
29        almostfull_C: cross cp_wr_en, cp_rd_en, cp_almostfull;
30        almostempty_C: cross cp_wr_en, cp_rd_en, cp_almostempty;
31        underflow_C: cross cp_wr_en, cp_rd_en, cp_underflow{
32            illegal_bins zero_r_one = binsof(cp_rd_en) intersect {0} && binsof(cp_underflow) intersect {1};
33        }
34    endgroup
35
36    function void sample_data(FIFO_transaction F_txn);
37        cvr_gp.sample();
38        F_cvg_txn = F_txn;
39    endfunction
40
41    function new();
42        cvr_gp = new();
43    endfunction
44
45 endclass
46 endpackage
47
```

4.0 DO FILE



```
1  vlib work
2  vlog *v +define+SIM +cover
3  vsim -voptargs=+acc top_module -cover
4  run 0
5  add wave *
6  add wave -position insertpoint \
7  sim:/top_module/f_if/clk \
8  sim:/top_module/f_if/data_in \
9  sim:/top_module/f_if/rst_n \
10 sim:/top_module/f_if/wr_en \
11 sim:/top_module/f_if/rd_en \
12 sim:/top_module/f_if/data_out \
13 sim:/top_module/f_if/wr_ack \
14 sim:/top_module/f_if/overflow \
15 sim:/top_module/f_if/underflow \
16 sim:/top_module/f_if/full \
17 sim:/top_module/f_if/empty \
18 sim:/top_module/f_if/almostfull \
19 sim:/top_module/f_if/almostempty
20 add wave -position insertpoint \
21 sim:/shared_pkg::correct_count \
22 sim:/shared_pkg::error_count \
23 sim:/shared_pkg::test_finished
24 coverage save top_module.ucdb -onexit -du FIFO
25 run -all
```

5.0 DUT

5.1 Without Edits

```
module FIFO(data_in, wr_en, rd_en, clk, rst_n, full, empty, almostfull,
almostempty, wr_ack, overflow, underflow, data_out);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
input [FIFO_WIDTH-1:0] data_in;
input clk, rst_n, wr_en, rd_en;
output reg [FIFO_WIDTH-1:0] data_out;
output reg wr_ack, overflow;
output full, empty, almostfull, almostempty, underflow;

localparam max_fifo_addr = $clog2(FIFO_DEPTH);

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= 0;
    end
    else if (wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= data_in;
        wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        wr_ack <= 0;
        if (full & wr_en)
            overflow <= 1;
        else
            overflow <= 0;
    end
end

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_ptr <= 0;
    end
    else if (rd_en && count != 0) begin
        data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
    end
end
```

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({wr_en, rd_en} == 2'b10) && !full)
            count <= count + 1;
        else if ( ({wr_en, rd_en} == 2'b01) && !empty)
            count <= count - 1;
    end
end

assign full = (count == FIFO_DEPTH)? 1 : 0;
assign empty = (count == 0)? 1 : 0;
assign underflow = (empty && rd_en)? 1 : 0;
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
assign almostempty = (count == 1)? 1 : 0;

endmodule

```

5.2 With Edits

5.2.1 Edits

```

1- reg [FIFO_WIDTH-1:0] mem [0:FIFO_DEPTH-1]; <- reg [FIFO_WIDTH-1:0] mem
[FIFO_DEPTH-1:0] (due to it is unpacked);
2- add this to write reset list (wr_ack <= 0; overflow <= 0; data_out <=
0;);
3- add this overflow <= 0; to
    ( else if (wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= data_in;
        wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;)
4- make underflow reg due to it is a seq signal
5- add this block since underflow is seq
(always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        underflow <= 0;
    else if (rd_en && empty)
        underflow <= 1;    // pulse on rejected read
    else
        underflow <= 0;
end)
6- if (rd_ptr == FIFO_DEPTH-1)
    rd_ptr <= 0;
    else (to be a wrap up pinter)
7- if (wr_ptr == FIFO_DEPTH-1) begin
    wr_ptr <= 0;
    end (to be a wrap up pinter)

```

```

8- assign almostfull = (count == FIFO_DEPTH-1)? 1 : 0; <- assign almostfull =
(count == FIFO_DEPTH-2)? 1 : 0;
9- else if ({wr_en, rd_en} == 2'b11) begin
    if (empty) begin
        count <= count + 1;
    end else if(full) begin
        count <= count - 1;
    end
    end    to handle when read and write is asserted
10- assign f_if.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;  change to -1
instead of -2

```

5.2.2 Code

```

module FIFO(fifo_if.DUT f_if);
    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    localparam max_fifo_addr = $clog2(FIFO_DEPTH);
    reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
    reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
    reg [max_fifo_addr:0] count;
    //FIFO_8
    `ifdef SIM
    always_comb begin
        if(!f_if.rst_n) begin
            count_rst: assert final(count == 0);
            wr_ptr_rst: assert final(wr_ptr == 0);
            rd_ptr_rst: assert final(rd_ptr == 0);
            count_rst_c: cover final(count == 0);
            wr_ptr_rst_c: cover final(wr_ptr == 0);
            rd_ptr_rst_c: cover final(rd_ptr == 0);

        end

        if (count == 1) begin
            alm_empty_check: assert final (f_if.almostempty == 1'b1);
        end else if(count == 0) begin
            empty_check_comb: assert final (f_if.empty == 1'b1);
        end else if (count == FIFO_DEPTH) begin
            full_check_comb: assert final (f_if.full == 1'b1);
        end else if(count == FIFO_DEPTH-1) begin
            alm_full_check: assert final (f_if.almostfull == 1'b1);
        end

    end

end

property reset_check;
    @(posedge f_if.clk) !f_if.rst_n |-> (wr_ptr == 0 && rd_ptr == 0 && count == 0);
endproperty

```



```

property wr_ack_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (f_if.wr_en && !f_if.full) | => f_if.wr_ack;
endproperty

property overflow_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (f_if.wr_en && f_if.full) | => f_if.overflow;
endproperty

property underflow_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (f_if.rd_en && f_if.empty) | => f_if.underflow;
endproperty

property empty_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (count == 0) | -> f_if.empty;
endproperty

property full_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (count == FIFO_DEPTH) | -> f_if.full;
endproperty

property almostfull_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (count == FIFO_DEPTH-1) | -> f_if.almostfull;
endproperty

property almostempty_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (count == 1) | -> f_if.almostempty;
endproperty

property wr_wrap_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (f_if.wr_en && !f_if.full && wr_ptr == FIFO_DEPTH-1) | => (wr_ptr == 0);
endproperty

property rd_wrap_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n)
    (f_if.rd_en && !f_if.empty && rd_ptr == FIFO_DEPTH-1) | => (rd_ptr == 0);
endproperty

property wr_threshold_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n) wr_ptr < FIFO_DEPTH;

```

```

endproperty

property rd_threshold_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n) rd_ptr < FIFO_DEPTH;
endproperty

property count_threshold_check;
  @(posedge f_if.clk) disable iff(!f_if.rst_n) count <= FIFO_DEPTH;
endproperty

property count_up_check;
  @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en) &&
  (!f_if.rd_en) && (!f_if.full) | => (count == $past(count) + 1'b1);
endproperty

property count_down_check;
  @(posedge f_if.clk) disable iff (!f_if.rst_n) (!f_if.wr_en) &&
  (f_if.rd_en) && (!f_if.empty) | => (count == $past(count) - 1'b1);
endproperty

property priority_check1;
  @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en) && (f_if.rd_en)
  && (f_if.empty) | => (count == $past(count) + 1'b1);
endproperty

property priority_check2;
  @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en) && (f_if.rd_en)
  && (f_if.full) | => (count == $past(count) - 1'b1);
endproperty

property wr_ptr_check;
  @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.wr_en) && (count <
  FIFO_DEPTH) | => (wr_ptr == $past(wr_ptr) + 1'b1);
endproperty

property rd_ptr_check;
  @(posedge f_if.clk) disable iff (!f_if.rst_n) (f_if.rd_en) && (count != 0)
  | => (rd_ptr == $past(rd_ptr) + 1'b1);
endproperty

a1:assert property(reset_check);
a2:assert property(wr_ack_check);
a3:assert property(overflow_check);
a4:assert property(underflow_check);
a5:assert property(empty_check);
a6:assert property(full_check);
a7:assert property(almostfull_check);
a8:assert property(almostempty_check);

```

```

a9:assert property(wr_wrap_check);
a10:assert property(rd_wrap_check);
a11:assert property(wr_threshold_check);
a12:assert property(rd_threshold_check);
a13:assert property(count_threshold_check);
a14:assert property(count_up_check);
a15:assert property(count_down_check);
a16:assert property(priority_check1);
a17:assert property(priority_check2);
a18:assert property(wr_ptr_check);
a19:assert property(rd_ptr_check);


c1:cover property(reset_check);
c2:cover property(wr_ack_check);
c3:cover property(overflow_check);
c4:cover property(underflow_check);
c5:cover property(empty_check);
c6:cover property(full_check);
c7:cover property(almostfull_check);
c8:cover property(almostempty_check);
c9:cover property(wr_wrap_check);
c10:cover property(rd_wrap_check);
c11:cover property(wr_threshold_check);
c12:cover property(rd_threshold_check);
c13:cover property(count_threshold_check);
c14:cover property(count_up_check);
c15:cover property(count_down_check);
c16:cover property(priority_check1);
c17:cover property(priority_check2);
c18:cover property(wr_ptr_check);
c19:cover property(rd_ptr_check);
`endif


always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        wr_ptr <= 0;
        f_if.wr_ack <= 0;
        f_if.overflow <= 0;
    end
    else if (f_if.wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= f_if.data_in;
        f_if.wr_ack <= 1;
        if (wr_ptr == FIFO_DEPTH-1) begin
            wr_ptr <= 0;
        end
        else begin
            wr_ptr <= wr_ptr + 1;
        end
    end
end

```

```

        f_if.overflow <= 0;
    end
    else begin
        f_if.wr_ack <= 0;
        if (f_if.full == 1'b1 && f_if.wr_en == 1'b1)
            f_if.overflow <= 1;
        else
            f_if.overflow <= 0;
        end
    end
end
always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        rd_ptr <= 0;
        f_if.data_out <= 0;
    end
    else if (f_if.rd_en && count != 0) begin
        f_if.data_out <= mem[rd_ptr];
        if (rd_ptr == FIFO_DEPTH-1)
            rd_ptr <= 0;
        else
            rd_ptr <= rd_ptr + 1;
        end
    end
end
always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
            count <= count + 1;
        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
            count <= count - 1;
        else if ( ({f_if.wr_en, f_if.rd_en} == 2'b11) begin
            if (f_if.empty) begin
                count <= count + 1;
            end else if (f_if.full) begin
                count <= count - 1;
            end
        end
    end
end
end
always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n)
        f_if.underflow <= 0;
    else if (f_if.rd_en && f_if.empty)
        f_if.underflow <= 1;
    else
        f_if.underflow <= 0;
end

```

```
end
assign f_if.full = (count == FIFO_DEPTH)? 1 : 0;
assign f_if.empty = (count == 0)? 1 : 0;
assign f_if.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
assign f_if.almostempty = (count == 1)? 1 : 0;

endmodule
```

6.0 SHARED PACKAGE



```
1 package shared_pkg;
2 integer correct_count,error_count;
3 bit test_finished;
4 event sample_event;
5 endpackage
```

7.0 TOP MODULE

```
1 module top_module();
2     bit clk;
3     initial begin
4         clk = 0;
5         forever begin
6             #1 clk=~clk;
7         end
8     end
9     fifo_if f_if (clk);
10    FIFO DUT (f_if);
11    fifo_tb TEST (f_if);
12    fifo_monitor MONITOR (f_if);
13
14    always_comb begin
15        if(!f_if.rst_n) begin
16            wr_ack_rst: assert final(f_if.wr_ack == 0);
17            overflow_rst: assert final(f_if.overflow == 0);
18            empty_rst: assert final(f_if.empty == 1);
19            underflow_rst: assert final(f_if.underflow == 0);
20            almostfull_rst: assert final(f_if.almostfull == 0);
21            almostempty_rst: assert final(f_if.almostempty == 0);
22            full_rst: assert final(f_if.full == 0);
23            wr_ack_rst_c: cover final(f_if.wr_ack == 0);
24            overflow_rst_c: cover final(f_if.overflow == 0);
25            empty_rst_c: cover final(f_if.empty == 1);
26            underflow_rst_c: cover final(f_if.underflow == 0);
27            almostfull_rst_c: cover final(f_if.almostfull == 0);
28            almostempty_rst_c: cover final(f_if.almostempty == 0);
29            full_rst_c: cover final(f_if.full == 0);
30        end
31    end
32
33 endmodule
```

8.0 TESTBENCH



```
1  import shared_pkg::*;
2  import FIFO_transaction_pkg::*;
3  module fifo_tb(fifo_if.TEST f_if);
4      FIFO_transaction tr = new(75,75);
5      initial begin
6          f_if.wr_en = 0; f_if.rd_en = 0;
7          f_if.data_in = 16'h0000;
8          //FIFO_5
9          assert_reset;
10         for (int i=0;i<15000;i++) begin
11             //FIFO_6
12             assert(tr.randomize());
13             f_if.rst_n = tr.rst_n;
14             f_if.data_in = tr.data_in;
15             f_if.wr_en = tr.wr_en;
16             f_if.rd_en = tr.rd_en;
17             @(negedge f_if.clk);
18             //FIFO_7
19             -> sample_event;
20         end
21         test_finished = 1;
22     end
23
24     task assert_reset;
25         f_if.rst_n = 0;
26         @(negedge f_if.clk);
27         f_if.rst_n = 1;
28         @(negedge f_if.clk);
29     endtask
30 endmodule
```

9.0 SCOREBOARD PACKAGE

```
1 package FIFO_scoreboard_pkg;
2 import shared_pkg::*;
3 import FIFO_transaction_pkg::*;
4
5 class FIFO_scoreboard;
6
7     bit [FIFO_WIDTH-1:0] data_out_ref;
8     reg [3:0] size;
9     logic [FIFO_WIDTH-1:0] fifo_mem[$];
10    int count = 0;
11    function new();
12        error_count = 0;
13        correct_count = 0;
14    endfunction
15
16    task check_data(FIFO_transaction F_txn);
17        reference_model(F_txn);
18        if (F_txn.data_out == data_out_ref) begin
19            correct_count++;
20        end else begin
21            $display("[WRONG] data_out mismatch at %0t: DUT=%h REF=%h",
22                    $time, F_txn.data_out, data_out_ref);
23            error_count++;
24        end
25    endtask
26
27    task reference_model(FIFO_transaction F_txn);
28        size = fifo_mem.size();
29        if(!F_txn.rst_n) begin
30            data_out_ref = 0;
31            fifo_mem.delete();
32            size = fifo_mem.size();
33        end
34        else begin
35            size = fifo_mem.size();
36            case ({F_txn.wr_en,F_txn.rd_en})
37                2'b10: begin
38                    if (size != 8) begin
39                        fifo_mem.push_back(F_txn.data_in);
40                        data_out_ref = data_out_ref;
41                    end
42                end
43            end
44
45            // Read only
46            2'b01: begin
47                if (size != 0) begin
48                    data_out_ref = fifo_mem.pop_front();
49                end
50            end
51
52            // Read + Write simultaneously
53            2'b11: begin
54                if (size == 0) begin
55                    fifo_mem.push_back(F_txn.data_in);
56                end
57                else if (size == 8) begin
58                    data_out_ref = fifo_mem.pop_front();
59                end
60                else begin
61                    data_out_ref = fifo_mem.pop_front();
62                    fifo_mem.push_back(F_txn.data_in);
63                end
64            end
65        endcase
66    endtask
67
68 endclass
69
70 endpackage
71
72
73
```


10.0 TRANSACTION

```
1 package FIFO_transaction_pkg;
2 import shared_pkg::*;
3 parameter FIFO_WIDTH = 16;
4 parameter FIFO_DEPTH = 8;
5 class FIFO_transaction;
6     rand bit [FIFO_WIDTH-1:0] data_in;
7     bit clk;
8     rand bit rst_n;
9     rand bit wr_en;
10    rand bit rd_en;
11    bit [FIFO_WIDTH-1:0] data_out;
12    bit wr_ack, overflow, underflow;
13    bit full, empty, almostfull, almostempty;
14    integer RD_EN_ON_DIST , WR_EN_ON_DIST;
15
16    function new(int rd_dist = 30, int wr_dist = 70);
17        this.RD_EN_ON_DIST = rd_dist;
18        this.WR_EN_ON_DIST = wr_dist;
19    endfunction
20    //FIFO_1
21    constraint rst_dist
22    {
23        rst_n dist {0:/5 , 1:/95};
24    }
25    //FIFO_2
26    constraint wr_en_dist
27    {
28        wr_en dist { 0:/(100 - WR_EN_ON_DIST) , 1:/WR_EN_ON_DIST};
29    }
30    //FIFO_3
31    constraint rd_en_c
32    {
33        rd_en dist { 0:/(100 - RD_EN_ON_DIST) , 1:/RD_EN_ON_DIST };
34    }
35 endclass
36
37 endpackage
38
```

11.0 INTERFACE

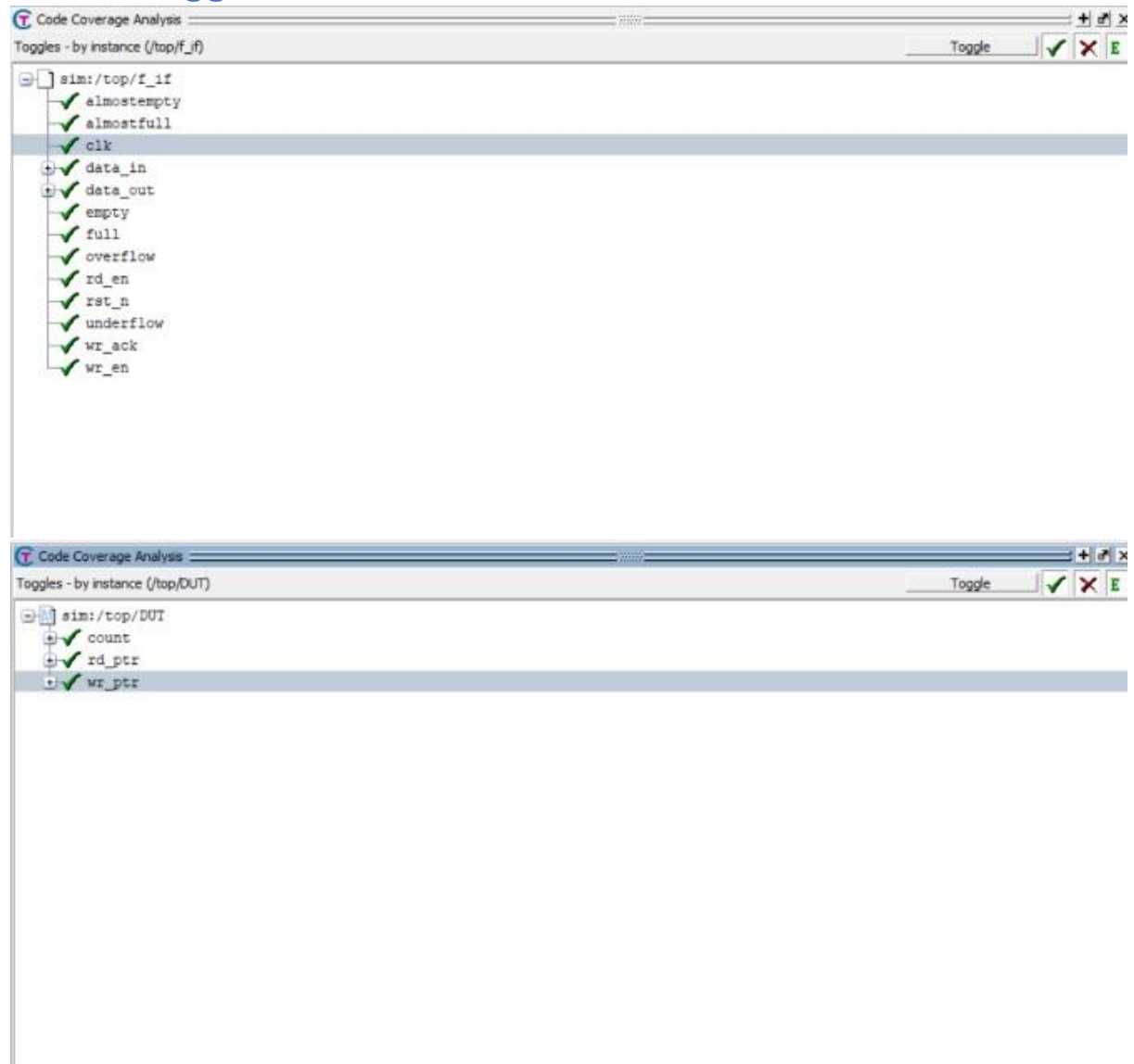
```
1 interface fifo_if(clk);
2     parameter FIFO_WIDTH = 16;
3     parameter FIFO_DEPTH = 8;
4     input bit clk;
5     logic [FIFO_WIDTH-1:0] data_in;
6     logic rst_n, wr_en, rd_en;
7     logic [FIFO_WIDTH-1:0] data_out;
8     logic wr_ack, overflow, underflow;
9     logic full, empty, almostfull, almostempty;
10
11     modport DUT (
12         input clk,
13         input rst_n,
14         input wr_en,
15         input rd_en,
16         input data_in,
17         output data_out,
18         output wr_ack,
19         output overflow,
20         output underflow,
21         output full,
22         output empty,
23         output almostfull,
24         output almostempty
25     );
26
27     modport TEST (
28         input clk,
29         output rst_n,
30         output wr_en,
31         output rd_en,
32         output data_in,
33         input data_out,
34         input wr_ack,
35         input overflow,
36         input underflow,
37         input full,
38         input empty,
39         input almostfull,
40         input almostempty
41     );
42
43     modport MONITOR (
44         input clk,
45         input rst_n,
46         input wr_en,
47         input rd_en,
48         input data_in,
49         input data_out,
50         input wr_ack,
51         input overflow,
52         input underflow,
53         input full,
54         input empty,
55         input almostfull,
56         input almostempty
57     );
58
59 endinterface
60
```

12.0 QUESTASIM

12.1 Transcript

```
# correct count = 15000 , error count = 0
```

12.2 Toggle



12.3 Branch

Code Coverage Analysis

Branches - by instance (/top/DUT)

Branch

```
228 else begin
230 if (f_if.full == 1'b1 && f_if.wr_en == 1'b1)
232 else
237 if (!f_if.rst_n) begin
241 else if (f_if.rd_en && count != 0) begin
243 if (rd_ptr == FIFO_DEPTH-1)
245 else
250 if (!f_if.rst_n) begin
253 else begin
254 if ( ([f_if.wr_en, f_if.rd_en] == 2'b10) && !f_if.full)
256 else if ( ([f_if.wr_en, f_if.rd_en] == 2'b01) && !f_if.empty)
258 else if ( ([f_if.wr_en, f_if.rd_en] == 2'b11) begin
259 if (f_if.empty) begin
261 end else if (f_if.full) begin
268 if (!f_if.rst_n)
270 else if (f_if.rd_en && f_if.empty)
272 else
275 assign f_if.full = (count == FIFO_DEPTH)? 1 : 0;
276 assign f_if.empty = (count == 0)? 1 : 0;
277 assign f_if.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
278 assign f_if.almostempty = (count == 1)? 1 : 0;
```

Code Coverage Analysis

Branches - by instance (/top/DUT)

Branch

```
code.av
53 if(!f_if.rst_n) begin
63 if (count == 1) begin
67 end else if (count == 0) begin
71 end else if (count == FIFO_DEPTH) begin
75 end else if (count == FIFO_DEPTH-1) begin
212 if (!f_if.rst_n) begin
217 else if (f_if.wr_en && count < FIFO_DEPTH) begin
220 if (wr_ptr == FIFO_DEPTH-1) begin
223 else begin
228 else begin
230 if (f_if.full == 1'b1 && f_if.wr_en == 1'b1)
232 else
237 if (!f_if.rst_n) begin
241 else if (f_if.rd_en && count != 0) begin
243 if (rd_ptr == FIFO_DEPTH-1)
245 else
250 if (!f_if.rst_n) begin
253 else begin
254 if ( ([f_if.wr_en, f_if.rd_en] == 2'b10) && !f_if.full)
256 else if ( ([f_if.wr_en, f_if.rd_en] == 2'b01) && !f_if.empty)
```

12.4 Statements

Code Coverage Analysis

Statements - by instance (/top/DUT)

Statement

```
code.av
52 always_comb begin
211 always @(posedge f_if.clk or negedge f_if.rst_n) begin
213 wr_ptr <= 0;
214 f_if.wr_ack <= 0;
215 f_if.overflow <= 0;
218 mem[wr_ptr] <= f_if.data_in;
219 f_if.wr_ack <= 1;
221 wr_ptr <= 0;
224 wr_ptr <= wr_ptr + 1;
226 f_if.overflow <= 0;
229 f_if.wr_ack <= 0;
231 f_if.overflow <= 1;
233 f_if.overflow <= 0;
236 always @(posedge f_if.clk or negedge f_if.rst_n) begin
238 rd_ptr <= 0;
239 f_if.data_out <= 0;
242 f_if.data_out <= mem[rd_ptr];
244 rd_ptr <= 0;
246 rd_ptr <= rd_ptr + 1;
249 always @(posedge f_if.clk or negedge f_if.rst_n) begin
```

Code Coverage Analysis

Statements - by instance (/top/DUT)

Statement

```

233 f_if.overflow <= 0;
236 always @(posedge f_if.clk or negedge f_if.rst_n) begin
238 rd_ptr <= 0;
239 f_if.data_out <= 0;
242 f_if.data_out <= mem[rd_ptr];
244 rd_ptr <= 0;
246 rd_ptr <= rd_ptr + 1;
249 always @(posedge f_if.clk or negedge f_if.rst_n) begin
251 count <= 0;
255 count <= count + 1;
257 count <= count - 1;
260 count <= count + 1;
262 count <= count - 1;
267 always @(posedge f_if.clk or negedge f_if.rst_n) begin
269 f_if.underflow <= 0;
271 f_if.underflow <= 1;
273 f_if.underflow <= 0;
275 assign f_if.full = (count == FIFO_DEPTH)? 1 : 0;
276 assign f_if.empty = (count == 0)? 1 : 0;
277 assign f_if.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
278 assign f_if.almostempty = (count == 1)? 1 : 0;

```

12.5 Cover Groups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comm
CROSS cvr_gp::full_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		8416	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		2754	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		54	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		2798	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		1	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		977	1	100.00...		✓			
illegal_bin one_f_one		0	-	-		✓			
CROSS cvr_gp::empty_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		470	1	100.00...		✓			
bin <auto[0],auto[1],auto[1]>		1407	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		131	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		229	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		7946	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		1347	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		2721	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		749	1	100.00...		✓			
CROSS cvr_gp::almostfull_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		169	1	100.00...		✓			
bin <auto[0],auto[1],auto[1]>		12	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		64	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		17	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		8247	1	100.00...		✓			
CVP cvr_gp::cp_underflow		100.00%	100	100.00...		✓			
bin auto[0]		13477	1	100.00...		✓			
bin auto[1]		1523	1	100.00...		✓			
CVP cvr_gp::cp_wr_ack		100.00%	100	100.00...		✓			
bin auto[0]		4370	1	100.00...		✓			
bin auto[1]		10630	1	100.00...		✓			
CROSS cvr_gp::wr_ack_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		7916	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		2714	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		500	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		2754	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		138	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		978	1	100.00...		✓			
illegal_bin zero_zero_one		0	-	-		✓			
CROSS cvr_gp::overflow_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		30	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		7	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		8386	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		2754	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		2845	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		978	1	100.00...		✓			
illegal_bin zero_wi_one		0	-	-		✓			
CROSS cvr_gp::full_C		100.00%	100	100.00...		✓			

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comm
bin <auto[1],auto[0],auto[1]>		64	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		17	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		8247	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		2742	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		2788	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		961	1	100.00...		✓			
CROSS cvr_gp::almostempty_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		3967	1	100.00...		✓			
bin <auto[0],auto[1],auto[1]>		545	1	100.00...		✓			
bin <auto[1],auto[0],auto[1]>		416	1	100.00...		✓			
bin <auto[0],auto[0],auto[1]>		288	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		4449	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		2209	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		2436	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		690	1	100.00...		✓			
CROSS cvr_gp::underflow_C		100.00%	100	100.00...		✓			
bin <auto[1],auto[1],auto[1]>		1127	1	100.00...		✓			
bin <auto[1],auto[1],auto[0]>		7289	1	100.00...		✓			
bin <auto[0],auto[1],auto[1]>		396	1	100.00...		✓			
bin <auto[0],auto[1],auto[0]>		2358	1	100.00...		✓			
bin <auto[1],auto[0],auto[0]>		2852	1	100.00...		✓			
bin <auto[0],auto[0],auto[0]>		978	1	100.00...		✓			
illegal_bin zero_1_one		0	-	-		✓			

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comm
/FIFO_coverage_pkg/FIFO_coverage		100.00%							
TyPB cvr_gp		100.00%	100	100.00...		✓		auto(1)	
CVP cvr_gp::cp_wr_en		100.00%	100	100.00...		✓			
bin auto[0]		3732	1	100.00...		✓			
bin auto[1]		11268	1	100.00...		✓			
CVP cvr_gp::cp_rd_en		100.00%	100	100.00...		✓			
bin auto[0]		3830	1	100.00...		✓			
bin auto[1]		11170	1	100.00...		✓			
CVP cvr_gp::cp_full		100.00%	100	100.00...		✓			
bin auto[0]		14945	1	100.00...		✓			
bin auto[1]		55	1	100.00...		✓			
CVP cvr_gp::cp_empty		100.00%	100	100.00...		✓			
bin auto[0]		12763	1	100.00...		✓			
bin auto[1]		2237	1	100.00...		✓			
CVP cvr_gp::cp_almostfull		100.00%	100	100.00...		✓			
bin auto[0]		14738	1	100.00...		✓			
bin auto[1]		262	1	100.00...		✓			
CVP cvr_gp::cp_almostempty		100.00%	100	100.00...		✓			
bin auto[0]		9784	1	100.00...		✓			
bin auto[1]		5216	1	100.00...		✓			
CVP cvr_gp::cp_overflow		100.00%	100	100.00...		✓			
bin auto[0]		14963	1	100.00...		✓			
bin auto[1]		37	1	100.00...		✓			

12.6 Cover Directives

Name	Language	Enabled	Log	Count	Atleast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak
/top/full_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	0
/top/DUT/count_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	0
/top/DUT/wr_ptr_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	0
/top/DUT/rd_ptr_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c1	SVA	✓	Off	789	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c2	SVA	✓	Off	10073	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c3	SVA	✓	Off	35	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c4	SVA	✓	Off	1445	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c5	SVA	✓	Off	2124	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c6	SVA	✓	Off	50	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c7	SVA	✓	Off	249	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c8	SVA	✓	Off	4947	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c9	SVA	✓	Off	961	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c10	SVA	✓	Off	817	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c11	SVA	✓	Off	14214	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c12	SVA	✓	Off	14214	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c13	SVA	✓	Off	14214	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c14	SVA	✓	Off	2561	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c15	SVA	✓	Off	2106	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c16	SVA	✓	Off	1066	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c17	SVA	✓	Off	29	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c18	SVA	✓	Off	10073	1	Unli...	1	100%		✓	0	0	0
/top/DUT/c19	SVA	✓	Off	8581	1	Unli...	1	100%		✓	0	0	0

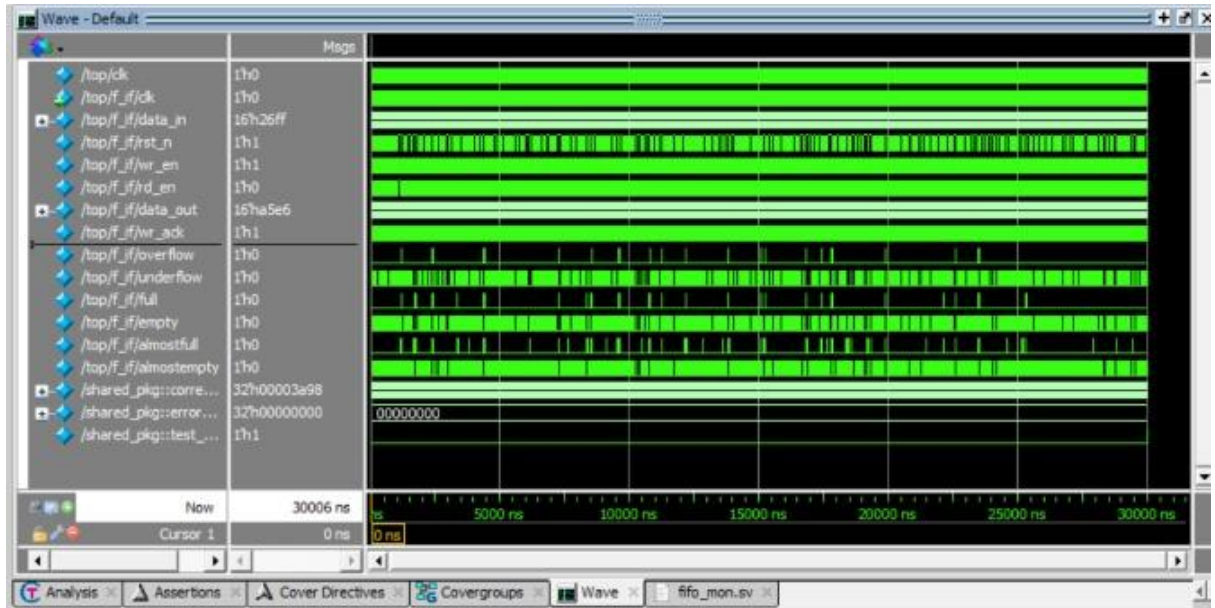
Cover Directives													
Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak
/top/wr_ack_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/overflow_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/empty_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/underflow_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/almostfull_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/almostempty_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/full_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/DUT/count_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/DUT/wr_ptr_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/DUT/rd_ptr_rst_c	SVA	✓	Off	749	1	Unli...	1	100%		✓	0	0	
/top/DUT/c1	SVA	✓	Off	789	1	Unli...	1	100%		✓	0	0	
/top/DUT/c2	SVA	✓	Off	10073	1	Unli...	1	100%		✓	0	0	
/top/DUT/c3	SVA	✓	Off	35	1	Unli...	1	100%		✓	0	0	
/top/DUT/c4	SVA	✓	Off	1445	1	Unli...	1	100%		✓	0	0	
/top/DUT/c5	SVA	✓	Off	2124	1	Unli...	1	100%		✓	0	0	
/top/DUT/c6	SVA	✓	Off	50	1	Unli...	1	100%		✓	0	0	
/top/DUT/c7	SVA	✓	Off	249	1	Unli...	1	100%		✓	0	0	
/top/DUT/c8	SVA	✓	Off	4947	1	Unli...	1	100%		✓	0	0	
/top/DUT/c9	SVA	✓	Off	961	1	Unli...	1	100%		✓	0	0	
/top/DUT/c10	SVA	✓	Off	817	1	Unli...	1	100%		✓	0	0	
/top/DUT/c11	SVA	✓	Off	14214	1	Unli...	1	100%		✓	0	0	
/top/DUT/c12	SVA	✓	Off	14214	1	Unli...	1	100%		✓	0	0	
/top/DUT/c13	SVA	✓	Off	14214	1	Unli...	1	100%		✓	0	0	

12.7 Assertions

Assertions									
Assertion Type	Language	Enable	Name	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time
Immediate	SVA	on	/top/DUT/empty_check_comb	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/full_check_comb	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/alm_full_check	0	1	-	-	-	-
Concurrent	SVA	on	/top/DUT/a1	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a2	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a3	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a4	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a5	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a6	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a7	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a8	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a9	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a10	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a11	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a12	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a13	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a14	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a15	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a16	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a17	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a18	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a19	0	1	-	08	08	0 ns
Immediate	SVA	on	/top/TEST/#anonbk#217929410#9...	0	1	-	-	-	-

Assertions									
Assertion Type	Language	Enable	Name	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time
Immediate	SVA	on	/top/wr_ack_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/overflow_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/empty_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/underflow_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/almostfull_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/almostempty_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/full_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/count_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/wr_ptr_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/rd_ptr_rst	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/alm_empty_check	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/empty_check_comb	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/full_check_comb	0	1	-	-	-	-
Immediate	SVA	on	/top/DUT/alm_full_check	0	1	-	-	-	-
Concurrent	SVA	on	/top/DUT/a1	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a2	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a3	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a4	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a5	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a6	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a7	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a8	0	1	-	08	08	0 ns
Concurrent	SVA	on	/top/DUT/a9	0	1	-	08	08	0 ns

12.8 Waveform



12.9 Coverage Report 1

Coverage Report by instance with details

```
=====
==== Instance: /top/DUT
==== Design Unit: work.FIFO
=====
```

Directive Coverage:

Directives	22	22	0	100.00%
------------	----	----	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/top/DUT/count_rst_c	FIFO		Verilog	SVA code.sv(58)	749	Covered
/top/DUT/wr_ptr_rst_c	FIFO		Verilog	SVA code.sv(59)	749	Covered
/top/DUT/rd_ptr_rst_c	FIFO		Verilog	SVA code.sv(60)	749	Covered
/top/DUT/c1	FIFO		Verilog	SVA code.sv(191)	789	Covered
/top/DUT/c2	FIFO		Verilog	SVA code.sv(192)	10073	Covered
/top/DUT/c3	FIFO		Verilog	SVA code.sv(193)	35	Covered
/top/DUT/c4	FIFO		Verilog	SVA code.sv(194)	1445	Covered
/top/DUT/c5	FIFO		Verilog	SVA code.sv(195)	2124	Covered

/top/DUT/c6	FIFO	Verilog	SVA	code.sv(196)	50 Covered
/top/DUT/c7	FIFO	Verilog	SVA	code.sv(197)	249 Covered
/top/DUT/c8	FIFO	Verilog	SVA	code.sv(198)	4947 Covered
/top/DUT/c9	FIFO	Verilog	SVA	code.sv(199)	961 Covered
/top/DUT/c10	FIFO	Verilog	SVA	code.sv(200)	817 Covered
/top/DUT/c11	FIFO	Verilog	SVA	code.sv(201)	14214 Covered
/top/DUT/c12	FIFO	Verilog	SVA	code.sv(202)	14214 Covered
/top/DUT/c13	FIFO	Verilog	SVA	code.sv(203)	14214 Covered
/top/DUT/c14	FIFO	Verilog	SVA	code.sv(204)	2561 Covered
/top/DUT/c15	FIFO	Verilog	SVA	code.sv(205)	2106 Covered
/top/DUT/c16	FIFO	Verilog	SVA	code.sv(206)	1066 Covered
/top/DUT/c17	FIFO	Verilog	SVA	code.sv(207)	29 Covered
/top/DUT/c18	FIFO	Verilog	SVA	code.sv(208)	10073 Covered
/top/DUT/c19	FIFO	Verilog	SVA	code.sv(209)	8581 Covered

```

=====
=====
=== Instance: /top
=== Design Unit: work.top
=====
=====

```

Directive Coverage:

Directives	7	7	0	100.00%
------------	---	---	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/top/wr_ack_rst_c	top	Verilog	SVA	top_module.sv(23)	749	Covered
/top/overflow_rst_c	top	Verilog	SVA	top_module.sv(24)	749	Covered
/top/empty_rst_c	top	Verilog	SVA	top_module.sv(25)	749	Covered
/top/underflow_rst_c	top	Verilog	SVA	top_module.sv(26)	749	Covered
/top/almostfull_rst_c	top	Verilog	SVA	top_module.sv(27)	749	Covered
/top/almostempty_rst_c	top	Verilog	SVA	top_module.sv(28)	749	Covered
/top/full_rst_c	top	Verilog	SVA	top_module.sv(29)	749	Covered

```

=====
Instance: /FIFO_coverage_pkg
Design Unit: work.FIFO_coverage_pkg
=====
=====

```

Covergroup Coverage:

```

Covergroups      1    na    na 100.00%
Coverpoints/Crosses 16    na    na    na
Covergroup Bins   66    66    0 100.00%

```

Covergroup	Metric	Goal	Bins	Status
TYPE /FIFO_coverage_pkg/FIFO_coverage/cvr_gp		100.00%	100	-
Covered				
covered/total bins:	66	66	-	
missing/total bins:	0	66	-	
% Hit:	100.00%	100	-	
Coverpoint cp_wr_en	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	3732	1	-	Covered
bin auto[1]	11268	1	-	Covered
Coverpoint cp_rd_en	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	3830	1	-	Covered
bin auto[1]	11170	1	-	Covered
Coverpoint cp_full	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	14945	1	-	Covered
bin auto[1]	55	1	-	Covered
Coverpoint cp_empty	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	12763	1	-	Covered
bin auto[1]	2237	1	-	Covered
Coverpoint cp_almostfull	100.00%	100	-	Covered
covered/total bins:	2	2	-	

missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	14738	1	-	Covered
bin auto[1]	262	1	-	Covered
Coverpoint cp_almostempty	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	9784	1	-	Covered
bin auto[1]	5216	1	-	Covered
Coverpoint cp_overflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	14963	1	-	Covered
bin auto[1]	37	1	-	Covered
Coverpoint cp_underflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	13477	1	-	Covered
bin auto[1]	1523	1	-	Covered
Coverpoint cp_wr_ack	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	4370	1	-	Covered
bin auto[1]	10630	1	-	Covered
Cross wr_ack_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	7916	1	-	Covered
bin <auto[1],auto[0],auto[1]>	2714	1	-	Covered
bin <auto[1],auto[1],auto[0]>	500	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2754	1	-	Covered
bin <auto[1],auto[0],auto[0]>	138	1	-	Covered
bin <auto[0],auto[0],auto[0]>	978	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin zero_zero_one	0		-	ZERO
Cross overflow_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				

bin <auto[1],auto[1],auto[1]>	30	1	-	Covered
bin <auto[1],auto[0],auto[1]>	7	1	-	Covered
bin <auto[1],auto[1],auto[0]>	8386	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2754	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2845	1	-	Covered
bin <auto[0],auto[0],auto[0]>	978	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin zero_w_one	0		-	ZERO
Cross full_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[0]>	8416	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2754	1	-	Covered
bin <auto[1],auto[0],auto[1]>	54	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2798	1	-	Covered
bin <auto[0],auto[0],auto[1]>	1	1	-	Covered
bin <auto[0],auto[0],auto[0]>	977	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin one_r_one	0		-	ZERO
Cross empty_C	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	470	1	-	Covered
bin <auto[0],auto[1],auto[1]>	1407	1	-	Covered
bin <auto[1],auto[0],auto[1]>	131	1	-	Covered
bin <auto[0],auto[0],auto[1]>	229	1	-	Covered
bin <auto[1],auto[1],auto[0]>	7946	1	-	Covered
bin <auto[0],auto[1],auto[0]>	1347	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2721	1	-	Covered
bin <auto[0],auto[0],auto[0]>	749	1	-	Covered
Cross almostfull_C	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	169	1	-	Covered
bin <auto[0],auto[1],auto[1]>	12	1	-	Covered
bin <auto[1],auto[0],auto[1]>	64	1	-	Covered
bin <auto[0],auto[0],auto[1]>	17	1	-	Covered
bin <auto[1],auto[1],auto[0]>	8247	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2742	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2788	1	-	Covered

bin <auto[0],auto[0],auto[0]>	961	1	-	Covered
Cross almostempty_C	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	3967	1	-	Covered
bin <auto[0],auto[1],auto[1]>	545	1	-	Covered
bin <auto[1],auto[0],auto[1]>	416	1	-	Covered
bin <auto[0],auto[0],auto[1]>	288	1	-	Covered
bin <auto[1],auto[1],auto[0]>	4449	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2209	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2436	1	-	Covered
bin <auto[0],auto[0],auto[0]>	690	1	-	Covered
Cross underflow_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	1127	1	-	Covered
bin <auto[1],auto[1],auto[0]>	7289	1	-	Covered
bin <auto[0],auto[1],auto[1]>	396	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2358	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2852	1	-	Covered
bin <auto[0],auto[0],auto[0]>	978	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin zero_r_one	0	-	ZERO	

COVERGROUP COVERAGE:

Covergroup	Metric	Goal	Bins	Status

TYPE /FIFO_coverage_pkg/FIFO_coverage/cvr_gp		100.00%	100	-
Covered				
covered/total bins:	66	66	-	
missing/total bins:	0	66	-	
% Hit:	100.00%	100	-	
Coverpoint cp_wr_en	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	3732	1	-	Covered
bin auto[1]	11268	1	-	Covered
Coverpoint cp_rd_en	100.00%	100	-	Covered
covered/total bins:	2	2	-	

missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	3830	1	-	Covered
bin auto[1]	11170	1	-	Covered
Coverpoint cp_full	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	14945	1	-	Covered
bin auto[1]	55	1	-	Covered
Coverpoint cp_empty	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	12763	1	-	Covered
bin auto[1]	2237	1	-	Covered
Coverpoint cp_almostfull	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	14738	1	-	Covered
bin auto[1]	262	1	-	Covered
Coverpoint cp_almostempty	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	9784	1	-	Covered
bin auto[1]	5216	1	-	Covered
Coverpoint cp_overflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	14963	1	-	Covered
bin auto[1]	37	1	-	Covered
Coverpoint cp_underflow	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	13477	1	-	Covered
bin auto[1]	1523	1	-	Covered
Coverpoint cp_wr_ack	100.00%	100	-	Covered
covered/total bins:	2	2	-	
missing/total bins:	0	2	-	
% Hit:	100.00%	100	-	
bin auto[0]	4370	1	-	Covered
bin auto[1]	10630	1	-	Covered

Cross wr_ack_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	7916	1	-	Covered
bin <auto[1],auto[0],auto[1]>	2714	1	-	Covered
bin <auto[1],auto[1],auto[0]>	500	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2754	1	-	Covered
bin <auto[1],auto[0],auto[0]>	138	1	-	Covered
bin <auto[0],auto[0],auto[0]>	978	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin zero_zero_one	0		-	ZERO

Cross overflow_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	30	1	-	Covered
bin <auto[1],auto[0],auto[1]>	7	1	-	Covered
bin <auto[1],auto[1],auto[0]>	8386	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2754	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2845	1	-	Covered
bin <auto[0],auto[0],auto[0]>	978	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin zero_w_one	0		-	ZERO

Cross full_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[0]>	8416	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2754	1	-	Covered
bin <auto[1],auto[0],auto[1]>	54	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2798	1	-	Covered
bin <auto[0],auto[0],auto[1]>	1	1	-	Covered
bin <auto[0],auto[0],auto[0]>	977	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin one_r_one	0		-	ZERO

Cross empty_C	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	470	1	-	Covered
bin <auto[0],auto[1],auto[1]>	1407	1	-	Covered

bin <auto[1],auto[0],auto[1]>	131	1	-	Covered
bin <auto[0],auto[0],auto[1]>	229	1	-	Covered
bin <auto[1],auto[1],auto[0]>	7946	1	-	Covered
bin <auto[0],auto[1],auto[0]>	1347	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2721	1	-	Covered
bin <auto[0],auto[0],auto[0]>	749	1	-	Covered
Cross almostfull_C	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	169	1	-	Covered
bin <auto[0],auto[1],auto[1]>	12	1	-	Covered
bin <auto[1],auto[0],auto[1]>	64	1	-	Covered
bin <auto[0],auto[0],auto[1]>	17	1	-	Covered
bin <auto[1],auto[1],auto[0]>	8247	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2742	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2788	1	-	Covered
bin <auto[0],auto[0],auto[0]>	961	1	-	Covered
Cross almostempty_C	100.00%	100	-	Covered
covered/total bins:	8	8	-	
missing/total bins:	0	8	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	3967	1	-	Covered
bin <auto[0],auto[1],auto[1]>	545	1	-	Covered
bin <auto[1],auto[0],auto[1]>	416	1	-	Covered
bin <auto[0],auto[0],auto[1]>	288	1	-	Covered
bin <auto[1],auto[1],auto[0]>	4449	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2209	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2436	1	-	Covered
bin <auto[0],auto[0],auto[0]>	690	1	-	Covered
Cross underflow_C	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin <auto[1],auto[1],auto[1]>	1127	1	-	Covered
bin <auto[1],auto[1],auto[0]>	7289	1	-	Covered
bin <auto[0],auto[1],auto[1]>	396	1	-	Covered
bin <auto[0],auto[1],auto[0]>	2358	1	-	Covered
bin <auto[1],auto[0],auto[0]>	2852	1	-	Covered
bin <auto[0],auto[0],auto[0]>	978	1	-	Covered
Illegal and Ignore Bins:				
illegal_bin zero_r_one	0	-	ZERO	

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
/top/wr_ack_rst_c	top	Verilog	SVA	top_module.sv(23)	749	Covered
/top/overflow_rst_c	top	Verilog	SVA	top_module.sv(24)	749	Covered
/top/empty_rst_c	top	Verilog	SVA	top_module.sv(25)	749	Covered
/top/underflow_rst_c	top	Verilog	SVA	top_module.sv(26)	749	Covered
/top/almostfull_rst_c	top	Verilog	SVA	top_module.sv(27)	749	Covered
/top/almostempty_rst_c	top	Verilog	SVA	top_module.sv(28)	749	Covered
/top/full_rst_c	top	Verilog	SVA	top_module.sv(29)	749	Covered
/top/DUT/count_rst_c	FIFO	Verilog	SVA	code.sv(58)	749	Covered
/top/DUT/wr_ptr_rst_c	FIFO	Verilog	SVA	code.sv(59)	749	Covered
/top/DUT/rd_ptr_rst_c	FIFO	Verilog	SVA	code.sv(60)	749	Covered
/top/DUT/c1	FIFO	Verilog	SVA	code.sv(191)	789	Covered
/top/DUT/c2	FIFO	Verilog	SVA	code.sv(192)	10073	Covered
/top/DUT/c3	FIFO	Verilog	SVA	code.sv(193)	35	Covered
/top/DUT/c4	FIFO	Verilog	SVA	code.sv(194)	1445	Covered
/top/DUT/c5	FIFO	Verilog	SVA	code.sv(195)	2124	Covered
/top/DUT/c6	FIFO	Verilog	SVA	code.sv(196)	50	Covered
/top/DUT/c7	FIFO	Verilog	SVA	code.sv(197)	249	Covered
/top/DUT/c8	FIFO	Verilog	SVA	code.sv(198)	4947	Covered
/top/DUT/c9	FIFO	Verilog	SVA	code.sv(199)	961	Covered
/top/DUT/c10	FIFO	Verilog	SVA	code.sv(200)	817	Covered
/top/DUT/c11	FIFO	Verilog	SVA	code.sv(201)	14214	Covered
/top/DUT/c12	FIFO	Verilog	SVA	code.sv(202)	14214	Covered
/top/DUT/c13	FIFO	Verilog	SVA	code.sv(203)	14214	Covered
/top/DUT/c14	FIFO	Verilog	SVA	code.sv(204)	2561	Covered
/top/DUT/c15	FIFO	Verilog	SVA	code.sv(205)	2106	Covered
/top/DUT/c16	FIFO	Verilog	SVA	code.sv(206)	1066	Covered
/top/DUT/c17	FIFO	Verilog	SVA	code.sv(207)	29	Covered
/top/DUT/c18	FIFO	Verilog	SVA	code.sv(208)	10073	Covered
/top/DUT/c19	FIFO	Verilog	SVA	code.sv(209)	8581	Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 29

Total Coverage By Instance (filtered view): 100.00%

12.10 Coverage Report 2

Coverage Report by instance with details

```
==== Instance: ^top_module#DUT
==== Design Unit: work.FIFO
=====
```

Assertion Coverage:

Assertions	26	26	0	100.00%
------------	----	----	---	---------

Name	File(Line)	Failure Count	Pass Count

^top_module#DUT /count_rst			
	code.sv(55)	0	1
^top_module#DUT /wr_ptr_rst			
	code.sv(56)	0	1
^top_module#DUT /rd_ptr_rst			
	code.sv(57)	0	1
^top_module#DUT /alm_empty_check			
	code.sv(64)	0	1
^top_module#DUT /empty_check_comb			
	code.sv(66)	0	1
^top_module#DUT /full_check_comb			
	code.sv(68)	0	1
^top_module#DUT /alm_full_check			
	code.sv(70)	0	1
^top_module#DUT /a1 code.sv(161)			
		0	1
^top_module#DUT /a2 code.sv(162)			
		0	1
^top_module#DUT /a3 code.sv(163)			
		0	1
^top_module#DUT /a4 code.sv(164)			
		0	1
^top_module#DUT /a5 code.sv(165)			
		0	1
^top_module#DUT /a6 code.sv(166)			
		0	1
^top_module#DUT /a7 code.sv(167)			
		0	1
^top_module#DUT /a8 code.sv(168)			
		0	1
^top_module#DUT /a9 code.sv(169)			
		0	1
^top_module#DUT /a10			
	code.sv(170)	0	1
^top_module#DUT /a11			
	code.sv(171)	0	1

```

^top_module#DUT /a12
    code.sv(172)          0    1
^top_module#DUT /a13
    code.sv(173)          0    1
^top_module#DUT /a14
    code.sv(174)          0    1
^top_module#DUT /a15
    code.sv(175)          0    1
^top_module#DUT /a16
    code.sv(176)          0    1
^top_module#DUT /a17
    code.sv(177)          0    1
^top_module#DUT /a18
    code.sv(178)          0    1
^top_module#DUT /a19
    code.sv(179)          0    1

```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	
Branches	39	39	0	100.00%

=====Branch
Details=====

Branch Coverage for instance ^top_module#DUT

Line	Item	Count	Source
----	----	-----	-----
File code.sv			
-----IF Branch-----			
54		15110	Count coming in to IF
54	1	1482	if(!f_ifrst_n) begin
		13628	All False Count

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
63		15110	Count coming in to IF
63	1	5199	if(count == 1) begin
65	1	2429	end else if(count == 0) begin
67	1	52	end else if(count == FIFO_DEPTH) begin
69	1	258	end else if(count == FIFO_DEPTH-1) begin
		7172	All False Count

Branch totals: 5 hits of 5 branches = 100.00%

-----IF Branch-----			
204		15752	Count coming in to IF

204	1	1538	if (!f_if.rst_n) begin
209	1	10631	else if (f_if.wr_en && count < FIFO_DEPTH)

begin

220	1	3583	else begin
-----	---	------	------------

Branch totals: 3 hits of 3 branches = 100.00%

-----IF Branch-----

212		10631	Count coming in to IF
212	1	1019	if (wr_ptr == FIFO_DEPTH-1) begin
215	1	9612	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

222		3583	Count coming in to IF
222	1	37	if (f_if.full == 1'b1 && f_if.wr_en == 1'b1)
224	1	3546	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

229		15752	Count coming in to IF
229	1	1538	if (!f_if.rst_n) begin
233	1	9037	else if (f_if.rd_en && count != 0) begin
		5177	All False Count

Branch totals: 3 hits of 3 branches = 100.00%

-----IF Branch-----

235		9037	Count coming in to IF
235	1	849	if (rd_ptr == FIFO_DEPTH-1)
237	1	8188	else

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

242		11948	Count coming in to IF
242	1	1520	if (!f_if.rst_n) begin
245	1	10428	else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----

246		10428	Count coming in to IF
246	1	2715	if (({f_if.wr_en, f_if.rd_en} == 2'b10) && !f_if.full)
248	1	2218	else if (({f_if.wr_en, f_if.rd_en} == 2'b01) && !f_if.empty)
250	1	4293	else if (({f_if.wr_en, f_if.rd_en} == 2'b11) begin

1202 All False Count

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
251          4293 Count coming in to IF
251      1      1127      if (f_if.empty) begin
253      1      30        end else if(f_if.full) begin
```

3136 All False Count

Branch totals: 3 hits of 3 branches = 100.00%

```
-----IF Branch-----
260          15752 Count coming in to IF
260      1      1538      if (!f_if.rst_n)
262      1      1524      else if (f_if.rd_en && f_if.empty)
264      1      12690      else
```

Branch totals: 3 hits of 3 branches = 100.00%

```
-----IF Branch-----
267          6763 Count coming in to IF
267      1      47      assign f_if.full = (count == FIFO_DEPTH)? 1 : 0;
267      2      6716      assign f_if.full = (count == FIFO_DEPTH)? 1 : 0;
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
268          6763 Count coming in to IF
268      1      1544      assign f_if.empty = (count == 0)? 1 : 0;
268      2      5219      assign f_if.empty = (count == 0)? 1 : 0;
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
269          6763 Count coming in to IF
269      1      106      assign f_if.almostfull = (count == FIFO_DEPTH-
1)? 1 : 0;
269      2      6657      assign f_if.almostfull = (count == FIFO_DEPTH-
1)? 1 : 0;
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
270          6763 Count coming in to IF
270      1      2089      assign f_if.almostempty = (count == 1)? 1 : 0;
270      2      4674      assign f_if.almostempty = (count == 1)? 1 : 0;
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage

Bins Covered Misses Coverage

```

-----
Conditions          26    26    0 100.00%
=====Condition
Details=====

```

Condition Coverage for instance ^top_module#DUT --

```

File code.sv
-----Focused Condition View-----
Line   63 Item   1 (count == 1)
Condition totals: 1 of 1 input term covered = 100.00%

```

```

Input Term  Covered Reason for no coverage Hint
-----
(count == 1)    Y

```

```

Rows:    Hits FEC Target      Non-masking condition(s)
-----
Row  1:    1 (count == 1)_0    -
Row  2:    1 (count == 1)_1    -

```

```

-----Focused Condition View-----
Line   65 Item   1 (count == 0)
Condition totals: 1 of 1 input term covered = 100.00%

```

```

Input Term  Covered Reason for no coverage Hint
-----
(count == 0)    Y

```

```

Rows:    Hits FEC Target      Non-masking condition(s)
-----
Row  1:    1 (count == 0)_0    -
Row  2:    1 (count == 0)_1    -

```

```

-----Focused Condition View-----
Line   67 Item   1 (count == 8)
Condition totals: 1 of 1 input term covered = 100.00%

```

```

Input Term  Covered Reason for no coverage Hint
-----
(count == 8)    Y

```

```

Rows:    Hits FEC Target      Non-masking condition(s)
-----
Row  1:    1 (count == 8)_0    -

```


Row 2: 1 (count == 8)_1 -

-----Focused Condition View-----

Line 69 Item 1 (count == (8 - 1))

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(count == (8 - 1))	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count == (8 - 1))_0	-
Row 2:	1	(count == (8 - 1))_1	-

-----Focused Condition View-----

Line 209 Item 1 (f_if.wr_en && (count < 8))

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
f_if.wr_en	Y		
(count < 8)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	f_if.wr_en_0	-
Row 2:	1	f_if.wr_en_1	(count < 8)
Row 3:	1	(count < 8)_0	f_if.wr_en
Row 4:	1	(count < 8)_1	f_if.wr_en

-----Focused Condition View-----

Line 212 Item 1 (wr_ptr == (8 - 1))

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(wr_ptr == (8 - 1))	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(wr_ptr == (8 - 1))_0	-
Row 2:	1	(wr_ptr == (8 - 1))_1	-

-----Focused Condition View-----

Line 222 Item 1 (f_if.full && f_if.wr_en)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

f_if.full	Y		
f_if.wr_en	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	f_if.full_0	-
Row 2:	1	f_if.full_1	f_if.wr_en
Row 3:	1	f_if.wr_en_0	f_if.full
Row 4:	1	f_if.wr_en_1	f_if.full

-----Focused Condition View-----

Line 233 Item 1 (f_if.rd_en && (count != 0))

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

f_if.rd_en	Y		
(count != 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	f_if.rd_en_0	-
Row 2:	1	f_if.rd_en_1	(count != 0)
Row 3:	1	(count != 0)_0	f_if.rd_en
Row 4:	1	(count != 0)_1	f_if.rd_en

-----Focused Condition View-----

Line 235 Item 1 (rd_ptr == (8 - 1))

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

(rd_ptr == (8 - 1))	Y		
---------------------	---	--	--

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(rd_ptr == (8 - 1))_0	-
Row 2:	1	(rd_ptr == (8 - 1))_1	-

-----Focused Condition View-----

Line 246 Item 1 ((~f_if.rd_en && f_if.wr_en) && ~f_if.full)

Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

f_if.rd_en	Y		
f_if.wr_en	Y		
f_if.full	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	f_if.rd_en_0	(~f_if.full && f_if.wr_en)
Row 2:	1	f_if.rd_en_1	-
Row 3:	1	f_if.wr_en_0	~f_if.rd_en
Row 4:	1	f_if.wr_en_1	(~f_if.full && ~f_if.rd_en)
Row 5:	1	f_if.full_0	(~f_if.rd_en && f_if.wr_en)
Row 6:	1	f_if.full_1	(~f_if.rd_en && f_if.wr_en)

-----Focused Condition View-----

Line 248 Item 1 ((f_if.rd_en && ~f_if.wr_en) && ~f_if.empty)

Condition totals: 3 of 3 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

f_if.rd_en	Y		
f_if.wr_en	Y		
f_if.empty	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	f_if.rd_en_0	-
Row 2:	1	f_if.rd_en_1	(~f_if.empty && ~f_if.wr_en)
Row 3:	1	f_if.wr_en_0	(~f_if.empty && f_if.rd_en)
Row 4:	1	f_if.wr_en_1	f_if.rd_en
Row 5:	1	f_if.empty_0	(f_if.rd_en && ~f_if.wr_en)
Row 6:	1	f_if.empty_1	(f_if.rd_en && ~f_if.wr_en)

-----Focused Condition View-----

Line 250 Item 1 (f_if.rd_en & f_if.wr_en)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

f_if.rd_en	Y		
f_if.wr_en	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
-------	------	------------	--------------------------

Row 1:	1	f_if.rd_en_0	f_if.wr_en
Row 2:	1	f_if.rd_en_1	f_if.wr_en
Row 3:	1	f_if.wr_en_0	f_if.rd_en
Row 4:	1	f_if.wr_en_1	f_if.rd_en

-----Focused Condition View-----

Line 262 Item 1 (f_if.rd_en && f_if.empty)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

f_if.rd_en	Y		
f_if.empty	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	f_if.rd_en_0	-
Row 2:	1	f_if.rd_en_1	f_if.empty
Row 3:	1	f_if.empty_0	f_if.rd_en
Row 4:	1	f_if.empty_1	f_if.rd_en

-----Focused Condition View-----

Line 267 Item 1 (count == 8)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

(count == 8)	Y		
--------------	---	--	--

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count == 8)_0	-
Row 2:	1	(count == 8)_1	-

-----Focused Condition View-----

Line 268 Item 1 (count == 0)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

(count == 0)	Y		
--------------	---	--	--

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count == 0)_0	-
Row 2:	1	(count == 0)_1	-

-----Focused Condition View-----

Line 269 Item 1 (count == (8 - 1))

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(count == (8 - 1))	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count == (8 - 1))_0	-
Row 2:	1	(count == (8 - 1))_1	-

-----Focused Condition View-----

Line 270 Item 1 (count == 1)

Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(count == 1)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count == 1)_0	-
Row 2:	1	(count == 1)_1	-

Directive Coverage:

Directives	22	22	0	100.00%
------------	----	----	---	---------

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status

^top_module#DUT /count_rst_c		FIFO	Verilog	SVA code.sv(58)	749	Covered
^top_module#DUT /wr_ptr_rst_c		FIFO	Verilog	SVA code.sv(59)	749	Covered
^top_module#DUT /rd_ptr_rst_c		FIFO	Verilog	SVA code.sv(60)	749	Covered
^top_module#DUT /c1		FIFO	Verilog	SVA code.sv(182)	789	Covered
^top_module#DUT /c2		FIFO	Verilog	SVA code.sv(183)	10073	Covered
^top_module#DUT /c3		FIFO	Verilog	SVA code.sv(184)	35	Covered

^top_module#DUT /c4	FIFO Verilog SVA code.sv(185)	1445 Covered
^top_module#DUT /c5	FIFO Verilog SVA code.sv(186)	2124 Covered
^top_module#DUT /c6	FIFO Verilog SVA code.sv(187)	50 Covered
^top_module#DUT /c7	FIFO Verilog SVA code.sv(188)	249 Covered
^top_module#DUT /c8	FIFO Verilog SVA code.sv(189)	4947 Covered
^top_module#DUT /c9	FIFO Verilog SVA code.sv(190)	961 Covered
^top_module#DUT /c10	FIFO Verilog SVA code.sv(191)	817 Covered
^top_module#DUT /c11	FIFO Verilog SVA code.sv(192)	14214 Covered
^top_module#DUT /c12	FIFO Verilog SVA code.sv(193)	14214 Covered
^top_module#DUT /c13	FIFO Verilog SVA code.sv(194)	14214 Covered
^top_module#DUT /c14	FIFO Verilog SVA code.sv(195)	2561 Covered
^top_module#DUT /c15	FIFO Verilog SVA code.sv(196)	2106 Covered
^top_module#DUT /c16	FIFO Verilog SVA code.sv(197)	1066 Covered
^top_module#DUT /c17	FIFO Verilog SVA code.sv(198)	29 Covered
^top_module#DUT /c18	FIFO Verilog SVA code.sv(199)	10073 Covered
^top_module#DUT /c19	FIFO Verilog SVA code.sv(200)	8581 Covered

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	
Statements	32	32	0	100.00%

=====Statement
Details=====

Statement Coverage for instance ^top_module#DUT --

Line	Item	Count	Source
----	----	-----	-----
File code.sv			
44			module FIFO(fifo_if.DUT f_if);
45			parameter FIFO_WIDTH = 16;
46			parameter FIFO_DEPTH = 8;
47			localparam max_fifo_addr = \$clog2(FIFO_DEPTH);
48			reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
49			reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
50			reg [max_fifo_addr:0] count;
51			//FIFO_8
52			`ifdef SIM
53			always_comb begin
54			if(!f_if.rst_n) begin
55			count_rst: assert final(count == 0);
56			wr_ptr_rst: assert final(wr_ptr == 0);
57			rd_ptr_rst: assert final(rd_ptr == 0);

```

58         count_rst_c: cover final(count == 0);
59         wr_ptr_rst_c: cover final(wr_ptr == 0);
60         rd_ptr_rst_c: cover final(rd_ptr == 0);
61
62         end
63         if(count == 1) begin
64             alm_empty_check: assert final (f_if.almostempty
== 1'b1);
65
66             end else if(count == 0) begin
67                 empty_check_comb: assert final (f_if.empty ==
1'b1);
68
69                 end else if (count == FIFO_DEPTH) begin
70                     full_check_comb: assert final (f_if.full == 1'b1);
71                     end else if(count == FIFO_DEPTH-1) begin
72                         alm_full_check: assert final (f_if.almostfull ==
1'b1);
73
74                         end
75
76                     end
77
78                 end
79
80             property reset_check;
81                 @(posedge f_if.clk) !f_if.rst_n |-> (wr_ptr == 0 &&
rd_ptr == 0 && count == 0);
82
83             endproperty
84
85             property wr_ack_check;
86                 @(posedge f_if.clk) disable iff(!f_if.rst_n)
(f_if.wr_en && !f_if.full) |=> f_if.wr_ack;
87
88             endproperty
89
90             property overflow_check;
91                 @(posedge f_if.clk) disable iff(!f_if.rst_n)
(f_if.wr_en && f_if.full) |=> f_if.overflow;
92
93             endproperty
94
95             property underflow_check;
96                 @(posedge f_if.clk) disable iff(!f_if.rst_n)
(f_if.rd_en && f_if.empty) |=> f_if.underflow;
97
98             endproperty
99
100            property empty_check;
101                @(posedge f_if.clk) disable iff(!f_if.rst_n)
(count == 0) |-> f_if.empty;
102
103            endproperty
104
105            property full_check;

```

```

100         @(posedge f_if.clk) disable iff(!f_if.rst_n)
101         (count == FIFO_DEPTH) |-> f_if.full;
102     endproperty
103
104     property almostfull_check;
105         @(posedge f_if.clk) disable iff(!f_if.rst_n)
106         (count == FIFO_DEPTH-1) |-> f_if.almostfull;
107     endproperty
108
109     property almostempty_check;
110         @(posedge f_if.clk) disable iff(!f_if.rst_n)
111         (count == 1) |-> f_if.almostempty;
112     endproperty
113
114     property wr_wrap_check;
115         @(posedge f_if.clk) disable iff(!f_if.rst_n)
116         (f_if.wr_en && !f_if.full && wr_ptr ==
FIFO_DEPTH-1) |=> (wr_ptr == 0);
117     endproperty
118
119     property rd_wrap_check;
120         @(posedge f_if.clk) disable iff(!f_if.rst_n)
121         (f_if.rd_en && !f_if.empty && rd_ptr ==
FIFO_DEPTH-1) |=> (rd_ptr == 0);
122     endproperty
123
124     property wr_threshold_check;
125         @(posedge f_if.clk) disable iff(!f_if.rst_n) wr_ptr <
FIFO_DEPTH;
126     endproperty
127
128     property rd_threshold_check;
129         @(posedge f_if.clk) disable iff(!f_if.rst_n) rd_ptr <
FIFO_DEPTH;
130     endproperty
131
132     property count_threshold_check;
133         @(posedge f_if.clk) disable iff(!f_if.rst_n) count <=
FIFO_DEPTH;
134     endproperty
135
136     property count_up_check;
137         @(posedge f_if.clk) disable iff(!f_if.rst_n)
(f_if.wr_en) && (!f_if.rd_en) && (!f_if.full) |=> (count == $past(count) + 1'b1);
138     endproperty
139

```



```

140                property count_down_check;
141                @(posedge f_if.clk) disable iff (!f_if.rst_n)
(!f_if.wr_en) && (f_if.rd_en) && (!f_if.empty) |>= (count == $past(count) - 1'b1);
142                endproperty
143
144                property priority_check1;
145                @(posedge f_if.clk) disable iff (!f_if.rst_n)
(f_if.wr_en) && (f_if.rd_en) && (f_if.empty) |>= (count == $past(count) + 1'b1);
146                endproperty
147
148                property priority_check2;
149                @(posedge f_if.clk) disable iff (!f_if.rst_n)
(f_if.wr_en) && (f_if.rd_en) && (f_if.full) |>= (count == $past(count) - 1'b1);
150                endproperty
151
152                property wr_ptr_check;
153                @(posedge f_if.clk) disable iff (!f_if.rst_n)
(f_if.wr_en) && (count < FIFO_DEPTH) |>= (wr_ptr == $past(wr_ptr) + 1'b1);
154                endproperty
155
156                property rd_ptr_check;
157                @(posedge f_if.clk) disable iff (!f_if.rst_n)
(f_if.rd_en) && (count != 0) |>= (rd_ptr == $past(rd_ptr) + 1'b1);
158                endproperty
159
160
161                a1:assert property(reset_check);
162                a2:assert property(wr_ack_check);
163                a3:assert property(overflow_check);
164                a4:assert property(underflow_check);
165                a5:assert property(empty_check);
166                a6:assert property(full_check);
167                a7:assert property(almostfull_check);
168                a8:assert property(almostempty_check);
169                a9:assert property(wr_wrap_check);
170                a10:assert property(rd_wrap_check);
171                a11:assert property(wr_threshold_check);
172                a12:assert property(rd_threshold_check);
173                a13:assert property(count_threshold_check);
174                a14:assert property(count_up_check);
175                a15:assert property(count_down_check);
176                a16:assert property(priority_check1);
177                a17:assert property(priority_check2);
178                a18:assert property(wr_ptr_check);
179                a19:assert property(rd_ptr_check);
180

```

```

181
182         c1:cover property(reset_check);
183         c2:cover property(wr_ack_check);
184         c3:cover property(overflow_check);
185         c4:cover property(underflow_check);
186         c5:cover property(empty_check);
187         c6:cover property(full_check);
188         c7:cover property(almostfull_check);
189         c8:cover property(almostempty_check);
190         c9:cover property(wr_wrap_check);
191         c10:cover property(rd_wrap_check);
192         c11:cover property(wr_threshold_check);
193         c12:cover property(rd_threshold_check);
194         c13:cover property(count_threshold_check);
195         c14:cover property(count_up_check);
196         c15:cover property(count_down_check);
197         c16:cover property(priority_check1);
198         c17:cover property(priority_check2);
199         c18:cover property(wr_ptr_check);
200         c19:cover property(rd_ptr_check);
201     `endif
202
203     1      15752    always @(posedge f_if.clk or negedge f_if.rst_n)
begin
204         if (!f_if.rst_n) begin
205             1      1538        wr_ptr <= 0;
206             1      1538        f_if.wr_ack  <= 0;
207             1      1538        f_if.overflow <= 0;
208         end
209         else if (f_if.wr_en && count < FIFO_DEPTH)
begin
210             1      10631       mem[wr_ptr] <= f_if.data_in;
211             1      10631       f_if.wr_ack <= 1;
212             if (wr_ptr == FIFO_DEPTH-1) begin
213                 1      1019        wr_ptr <= 0;
214             end
215             else begin
216                 1      9612        wr_ptr <= wr_ptr + 1;
217             end
218             1      10631       f_if.overflow <= 0;
219         end
220         else begin
221             1      3583        f_if.wr_ack <= 0;
222             if (f_if.full == 1'b1 && f_if.wr_en ==
1'b1)
223                 1      37        f_if.overflow <= 1;

```

```

224                                     else
225         1           3546           f_if.overflow <= 0;
226                                     end
227                                     end
228         1           15752      always @(posedge f_if.clk or negedge f_if.rst_n)
begin
229                                     if (!f_if.rst_n) begin
230         1           1538           rd_ptr <= 0;
231         1           1538           f_if.data_out <= 0;
232                                     end
233                                     else if (f_if.rd_en && count != 0) begin
234         1           9037           f_if.data_out <= mem[rd_ptr];
235                                     if (rd_ptr == FIFO_DEPTH-1)
236         1           849           rd_ptr <= 0;
237                                     else
238         1           8188           rd_ptr <= rd_ptr + 1;
239                                     end
240                                     end
241         1           11948      always @(posedge f_if.clk or negedge f_if.rst_n)
begin
242                                     if (!f_if.rst_n) begin
243         1           1520           count <= 0;
244                                     end
245                                     else begin
246                                     if      ( ({f_if.wr_en, f_if.rd_en} ==
2'b10) && !f_if.full)
247         1           2715           count <= count + 1;
248                                     else if ( ({f_if.wr_en, f_if.rd_en} ==
2'b01) && !f_if.empty)
249         1           2218           count <= count - 1;
250                                     else if ( {f_if.wr_en, f_if.rd_en} ==
2'b11) begin
251                                     if (f_if.empty) begin
252         1           1127           count <= count + 1;
253                                     end else if (f_if.full) begin
254         1           30           count <= count - 1;
255                                     end
256                                     end
257                                     end
258                                     end
259         1           15752      always @(posedge f_if.clk or negedge f_if.rst_n)
begin
260                                     if (!f_if.rst_n)
261         1           1538           f_if.underflow <= 0;
262                                     else if (f_if.rd_en && f_if.empty)
263         1           1524           f_if.underflow <= 1;

```

```

264                                     else
265         1          12690      f_if.underflow <= 0;
266                                     end
267         1          6764      assign f_if.full = (count == FIFO_DEPTH)? 1 : 0;
268         1          6764      assign f_if.empty = (count == 0)? 1 : 0;
269         1          6764      assign f_if.almostfull = (count == FIFO_DEPTH-
1)? 1 : 0;
270         1          6764      assign f_if.almostempty = (count == 1)? 1 : 0;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	
Toggles	20	20	0	100.00%

=====Toggle
Details=====

Toggle Coverage for instance ^top_module#DUT --

Node	1H->0L	0L->1H	"Coverage"
count[3-0]	1	1	100.00
rd_ptr[2-0]	1	1	100.00
wr_ptr[2-0]	1	1	100.00

Total Node Count = 10
Toggled Node Count = 10
Untoggled Node Count = 0

Toggle Coverage = 100.00% (20 of 20 bins)

DIRECTIVE COVERAGE:

Name	Design Unit	Design UnitType	Lang	File(Line)	Hits	Status
^top_module#DUT /count_rst_c			FIFO	Verilog SVA code.sv(58)	749	Covered
^top_module#DUT /wr_ptr_rst_c			FIFO	Verilog SVA code.sv(59)	749	Covered
^top_module#DUT /rd_ptr_rst_c			FIFO	Verilog SVA code.sv(60)	749	Covered
^top_module#DUT /c1			FIFO	Verilog SVA code.sv(182)	789	Covered
^top_module#DUT /c2			FIFO	Verilog SVA code.sv(183)	10073	Covered
^top_module#DUT /c3			FIFO	Verilog SVA code.sv(184)	35	Covered

^top_module#DUT /c4	FIFO	Verilog	SVA	code.sv(185)	1445 Covered
^top_module#DUT /c5	FIFO	Verilog	SVA	code.sv(186)	2124 Covered
^top_module#DUT /c6	FIFO	Verilog	SVA	code.sv(187)	50 Covered
^top_module#DUT /c7	FIFO	Verilog	SVA	code.sv(188)	249 Covered
^top_module#DUT /c8	FIFO	Verilog	SVA	code.sv(189)	4947 Covered
^top_module#DUT /c9	FIFO	Verilog	SVA	code.sv(190)	961 Covered
^top_module#DUT /c10	FIFO	Verilog	SVA	code.sv(191)	817 Covered
^top_module#DUT /c11	FIFO	Verilog	SVA	code.sv(192)	14214 Covered
^top_module#DUT /c12	FIFO	Verilog	SVA	code.sv(193)	14214 Covered
^top_module#DUT /c13	FIFO	Verilog	SVA	code.sv(194)	14214 Covered
^top_module#DUT /c14	FIFO	Verilog	SVA	code.sv(195)	2561 Covered
^top_module#DUT /c15	FIFO	Verilog	SVA	code.sv(196)	2106 Covered
^top_module#DUT /c16	FIFO	Verilog	SVA	code.sv(197)	1066 Covered
^top_module#DUT /c17	FIFO	Verilog	SVA	code.sv(198)	29 Covered
^top_module#DUT /c18	FIFO	Verilog	SVA	code.sv(199)	10073 Covered
^top_module#DUT /c19	FIFO	Verilog	SVA	code.sv(200)	8581 Covered

TOTAL DIRECTIVE COVERAGE: 100.00% COVERS: 22

ASSERTION RESULTS:

Name	File(Line)	Failure Count	Pass Count
^top_module#DUT /count_rst	code.sv(55)	0	1
^top_module#DUT /wr_ptr_rst	code.sv(56)	0	1
^top_module#DUT /rd_ptr_rst	code.sv(57)	0	1
^top_module#DUT /alm_empty_check	code.sv(64)	0	1
^top_module#DUT /empty_check_comb	code.sv(66)	0	1
^top_module#DUT /full_check_comb	code.sv(68)	0	1
^top_module#DUT /alm_full_check	code.sv(70)	0	1
^top_module#DUT /a1	code.sv(161)	0	1
^top_module#DUT /a2	code.sv(162)	0	1
^top_module#DUT /a3	code.sv(163)	0	1
^top_module#DUT /a4	code.sv(164)	0	1
^top_module#DUT /a5	code.sv(165)	0	1

^top_module#DUT /a6 code.sv(166)	0	1
^top_module#DUT /a7 code.sv(167)	0	1
^top_module#DUT /a8 code.sv(168)	0	1
^top_module#DUT /a9 code.sv(169)	0	1
^top_module#DUT /a10		
code.sv(170)	0	1
^top_module#DUT /a11		
code.sv(171)	0	1
^top_module#DUT /a12		
code.sv(172)	0	1
^top_module#DUT /a13		
code.sv(173)	0	1
^top_module#DUT /a14		
code.sv(174)	0	1
^top_module#DUT /a15		
code.sv(175)	0	1
^top_module#DUT /a16		
code.sv(176)	0	1
^top_module#DUT /a17		
code.sv(177)	0	1
^top_module#DUT /a18		
code.sv(178)	0	1
^top_module#DUT /a19		
code.sv(179)	0	1

Total Coverage By Instance (filtered view): 100.00%